

XS

독립세션 · 강의본

자율주행 포트폴리오

에이전트 아키텍처로 자산배분을 자동화한다

Ang · Azimbayev · Kim (2026)의 파이프라인을
우리 16주 교재와 Claude Code로 구현한다

운용역의 일이 감독으로 바뀐다면

무엇을 감독하는가

논문이 말하는 전환

- 에이전틱 AI는 투자자의 역할을 분석 실행에서 감독으로 옮긴다
- 약 50개 에이전트가 가정을 세우고 포트폴리오를 짜고 서로를 비평한다
- 메타 에이전트는 자기 지시문을 고쳐 쓴다

그래서 남는 질문

- 무엇을 근거로 이 안을 승인하는가
- 규칙을 어긴 안이 성과가 좋으면 규칙을 바꾸는가
- 책임은 누구에게 남는가

이 세션의 목표 — 파이프라인을 직접 만들어 돌려 보고, 감독해야 할 지점을 손으로 짚는다

코딩 경험은 필요하지 않다. 모든 실습은 [어디] [입력] [출력] [확인] 네 칸으로 적었다.

The Self Driving Portfolio

Agentic Architecture for Institutional Asset Management

항목	내용
저자	Andrew Ang · Nazym Azimbayev · Andrey Kim
출처	arXiv 2604.02279 · SSRN 6504386 (2026-04-02)
분량	31쪽 · 전시 11개
분류	AI · 멀티에이전트 시스템 · 포트폴리오 운용
규모	전문 에이전트 약 50개 · 포트폴리오 구성 방법 20가지 이상

"에이전틱 AI는 투자자의 역할을 분석 실행에서 감독으로 옮긴다" — 초록 첫 문장

Andrew Ang은 팩터투자 연구자이자 BlackRock 전 매니징디렉터다. 이 논문은 그가 실무에서 본 자산배분 절차를 에이전트로 옮긴 결과다.

파이프라인은 여섯 층이다

위에서 아래로 흐르고, 마지막 층이 위층을 고쳐 쓴다

층	하는 일	우리 교재
거버넌스	IPS가 전 과정을 제약한다	W01 · W16
입력	자본시장 가정(CMA)을 만든다	W03
구성	20가지 넘는 방법이 경쟁한다	W04 · W05 · W06
비평·투표	서로의 안을 심사하고 표결한다	16주 IC 전체
탐색	리서처가 없는 방법을 제안한다	W13 · W14
자기개선	메타가 코드와 프롬프트를 고쳐 쓴다	W10

여섯 층 모두 우리가 이미 가르친 내용이다 — 빠진 것은 엮는 방법뿐이다

입력 — 자본시장 가정

무엇을 믿고 배분하는가

논문에서

- 여러 CMA 에이전트가 서로 다른 방법으로 기대수익률을 만든다
- 단일 전망에 의존하지 않고 분포를 본다
- 가정의 출처와 방법을 산출물에 남긴다

W03에서 배운 것

- 빌딩블록 방식 — 실질금리 + 인플레이 + 위험프리미엄
- 기관마다 CMA가 다른 이유와 그 폭
- 가정이 배분을 지배한다

핵심 — 가정을 숨기지 않는다. 숨긴 가정은 감독할 수 없다

MVP에서는 역사적 평균을 그랜드 평균 쪽으로 축소하고 공분산에 Ledoit-Wolf 축소를 적용한다. 축소 비율은 명시적 가정으로 기록한다.

구성 — 스무 가지 방법이 경쟁한다

하나의 최적해가 아니라 여러 후보

방법	무엇에 의존하는가	약점	교재
MVO	기대수익률과 공분산	추정오차에 민감 · 코너해	W04
블랙리터맨	시장균형 + 뷰	뷰의 강도 설정이 자의적	W05
리스크 패리티	공분산만 (기대수익률 불필요)	수익률 목표를 직접 겨냥하지 않음	W06

약점이 서로 다르다는 것이 경쟁시키는 이유다

논문은 20가지 넘는 방법을 병렬로 돌린다. 우리 MVP는 교재에서 배운 세 가지만 쓴다. 총을 줄이지 않고 각 층의 폭만 줄이는 것이 축소의 원칙이다.

비평과 투표 — 이것이 우리 IC다

16주 동안 사람으로 해온 일을 그대로 옮긴 총

우리가 매주 4교시에 해온 절차

- 안을 제출한다 — 근거와 가정을 함께 낸다
- 정책 위반을 먼저 가린다 — 위반은 표결에 올리지 않는다
- 서로 다른 관점에서 심사한다 — 한 잣대로 줄세우지 않는다
- 표결하고 의결문을 남긴다 — 표결 자체가 아니라 기록이 산출물이다

논문의 critic·voting 총은 이 절차의 자동화다 — 새 개념이 아니다

그래서 이 세션은 새로운 것을 배우는 자리가 아니다. 해온 일을 기계에 옮기면 무엇이 남고 무엇이 사라지는지 확인하는 자리다. 사라지는 것 가운데 남겨야 할 것이 감독의 대상이 된다.

탐색 — 아직 없는 방법을 제안한다

리서치 에이전트

무엇을 하는가

- 현재 파이프라인에 없는 구성 방법을 찾는다
- 문헌과 기존 산출물을 대조해 빈틈을 짚는다
- 제안일 뿐 채택은 표결을 거친다

우리 교재와의 접점

- W13 팩터투자 — 어떤 팩터를 추가할 것인가
- W14 매크로·CTA — 국면 전환을 반영할 것인가
- 새 방법의 검증 부담은 제안자에게 있다

MVP에서는 구현하지 않는다 — 여섯 층 가운데 유일하게 생략한 층이다

이유는 검증 비용이다. 새 방법을 제안받아도 그것이 타당한지 가릴 절차가 먼저 서야 한다. MVP는 그 절차(비평·투표)를 먼저 세우는 데 집중한다.

자기개선 — 스스로를 고쳐 쓴다

이 논문에서 가장 논쟁적인 층

메타 에이전트가 하는 일

- 과거 예측을 실현 수익률과 대조한다
- 어느 에이전트가 어디서 얼마나 틀렸는지 자산별로 남긴다
- 다른 에이전트의 코드와 프롬프트를 고쳐 쓴다

여기서 감독이 필요하다 — 무엇을 근거로 지시문이 바뀌었는지 사람이 알 수 있는가

한 구간의 성과가 나빴다는 사실만으로 방법을 바꾸면, 잡음을 좇아 지시문이 흔들린다. 우리 MVP는 제안까지만 하고 반영은 사람이 결정하도록 IPS에 못박았다. 이 선택이 옳은지는 이 세션의 케이스에서 다룬다.

거버넌스 — 투자정책서가 통제한다

사람을 규율하던 문서가 기계를 규율한다

논문의 결론 문장

"전체 파이프라인은 투자정책서가 통제한다 — 사람 운용역을 인도하던 바로 그 문서가 이제 자율 에이전트를 제약하고 지시할 수 있다."

IPS 조항

파이프라인에서의 작동

자산배분 범위

최적화 문제의 제약식으로 들어간다

분산 요건

유효 종목 수 하한 — 코너해를 자동 기각한다

감독 조항

산출물은 권고안 · 집행은 사람 승인

킬스위치

지시문 자동 반영을 금지한다

우리 16주가 곧 이 파이프라인이다

각 층은 이미 배운 주차의 내용이다

층	주차	그 주에 배운 것	파이프라인에서의 역할
입력	W03	CMA 빌딩블록	기대수익률·공분산 산출
구성	W04	MVO와 공분산 추정	후보 ① 최대 샤프
구성	W05	블랙리터맨	후보 ② 균형 + 뷰
구성	W06	리스크 패리티·HRP	후보 ③ 위험기여 균등
비평	IC	모의 투자위원회 16회	적격 심사와 표결
개선	W10	위험관리와 성과평가	예측 대 실현 대조
통제	W16	TPA 총자산 접근	IPS와 감독 체계

이 표가 이 세션의 설계도다 — 새로 배울 이론은 없다

실습 데이터를 새로 만들지 않는다

주차별 실습데이터 덱과 CSV를 그대로 쓴다

이미 있는 것

- 각 주차 폴더의 실습데이터 덱 — 명세와 수집 프롬프트
- 가상 데이터 CSV 7개 — 부채·PE·채권·KFP
- 공개 데이터 수집 요령 — yfinance · FRED

이 세션에서 쓰는 것

- W04 명세의 다자산 월간 총수익률 패널
- 자산 11개 · 120개월 (2015-08 ~ 2025-07)
- 가상 기금 KFP — W16 케이스의 그 기금

재사용 사슬 — W02 패널 → W04 공분산 → W05 BL → W06 ERC → W10 성과평가 → W16 TPA

이 사슬이 그대로 파이프라인의 데이터 흐름이 된다.

구조는 harness로 세운다

에이전트 팀을 설계하는 메타 스킬

항목	내용
저장소	github.com/revfactory/harness (황민호)
라이선스	Apache-2.0 — 강의·상업 이용 모두 가능
무엇인가	도메인에 맞는 에이전트 팀을 설계하고, 각 역할을 정의하고, 그들이 쓸 스킬을 생성하는 Claude Code 스킬
왜 이것인가	논문의 여섯 층과 개념이 1대 1로 맞는다

메타 스킬이라는 말의 뜻

- 스킬이 일을 한다면, 메타 스킬은 일할 스킬과 에이전트를 만든다
- 우리는 포트폴리오를 짜는 코드를 직접 쓰지 않는다 — 짜 팀을 설계한다

harness는 여덟 단계로 팀을 세운다

우리는 이 가운데 여섯 단계를 쓴다

Phase	하는 일	이번 세션
0 현황 감사	기존 에이전트·스킬을 먼저 읽는다	빈 폴더에서 시작
1 도메인 분석	작업 유형을 식별한다	자산배분 파이프라인
2 팀 아키텍처	실행 모드와 패턴을 고른다	하이브리드
3 에이전트 정의	.claude/agents/{이름}.md 생성	7개
4 스킬 생성	.claude/skills/{이름}/SKILL.md	오케스트레이터 1개
5 통합·오케스트레이션	순서와 데이터 전달을 정한다	6단계 파이프라인
6 검증	구조와 트리거를 확인한다	필수 섹션 점검
7 진화	실행 후 피드백을 반영한다	메타 에이전트가 담당

여섯 패턴 가운데 셋을 골랐다

층마다 협업 방식이 다르다

패턴	언제 쓰는가	우리 파이프라인에서
파이프라인	순차 의존 작업	CMA → 배분 → 표결 → 리뷰
팬아웃/팬인	병렬 독립 작업	세 배분 에이전트를 동시에
생성-검증	생성 후 품질 검수	배분안 → IC 심사
전문가 풀	상황별 선택 호출	쓰지 않음 — 모두 항상 돌린다
감독자	중앙이 상태를 관리	쓰지 않음 — 파일로 주고받는다
계층적 위임	재귀적 하위 위임	쓰지 않음 — MVP에는 과하다

쓰지 않은 패턴을 적어 두는 것도 설계다 — 왜 쓰지 않았는지가 근거가 된다

논문의 층과 harness의 단계

같은 문제를 같은 방향으로 풀었다

논문	harness
IPS가 파이프라인을 통제	Phase 5-4 — CLAUDE.md에 포인터와 트리거 규칙 등록
약 50개 전문 에이전트	Phase 2-3 분리 기준 → Phase 3 에이전트 정의 파일
CMA 병렬 생성	Phase 2-2 팬아웃/팬인
20가지 방법 경쟁	Phase 2-2 전문가 풀
비평가 투표	Phase 2-2 생성-검증
메타 에이전트의 자기 재작성	Phase 7 하네스 진화 (피드백 → 반영 → 이력)

harness의 첫 번째 원칙은 "하네스는 고정물이 아니라 진화하는 시스템이다"이다. 논문의 메타 에이전트가 말하는 바와 같다.

에이전트는 파일 하나로 정의된다

프롬프트에 역할을 적어 넘기지 않는다

.claude/agents/cma-builder.md (발췌)

```
---
name: cma-builder
description: 자본시장 가정(CMA)을 생성한다.
  기대수익률과 공분산 행렬이 필요할 때 호출한다.
---

## 작업 원칙
1. 평가구간을 보지 않는다. SPLIT 이후 데이터는
  어떤 이유로도 참조하지 않는다.
2. 역사적 평균을 그대로 쓰지 않는다.
3. 가정을 숨기지 않는다.
```

필수 섹션

- 핵심 역할
- 작업 원칙
- 입력/출력 프로토콜
- 에러 핸들링
- 협업
- 팀 통신 프로토콜

파일로 두는 이유 — 버전 관리가 되고, 사람이 읽고 고칠 수 있다

오케스트레이터가 순서를 정한다

누가 언제 어떤 순서로 협업하는가

단계	에이전트	패턴	산출
1	cma-builder	파이프라인	cma.json
2	alloc-mvo · alloc-bl · alloc-riskparity	팬아웃	alloc_*.json
3	ic-critic	팬인 + 생성-검증	ic_vote.json
4	meta-reviewer	파이프라인	meta_review.json

데이터 전달 규칙

- 에이전트끼리 값을 직접 넘기지 않는다 — 모두 runs/{날짜}/의 JSON을 거친다
- 이유는 감사 가능성이다. 어느 단계의 무엇이 다음에 들어갔는지 파일로 남는다
- IPS 7.2항(근거를 남기지 않은 안은 무효)이 이 설계를 요구한다

정책 문서가 시스템에 꽂히는 자리

CLAUDE.md는 새 세션마다 읽힌다

CLAUDE.md

자율주행 포트폴리오 MVP

하네스

트리거: 자산배분·포트폴리오 구성·CMA·IC 표결 관련 작업 요청 시
self-driving-portfolio 스킬을 사용하라.

- 에이전트 7개: .claude/agents/
- 정책: ips.md — 이 파이프라인의 헌법이다. 에이전트는 이 문서의 제약 안에서만 산출물을 낸다. 행동을 바꾸려면 코드가 아니라 이 문서를 고친다.

논문의 주장이 여기서 구현된다 — 사람을 규율하던 문서가 그대로 기계를 규율한다

신 개를 일곱 개로 줄인다

층은 모두 지나되 각 층의 폭만 줄인다

#	에이전트	층	교재
0	ips-guardian	거버넌스 — 정책 해석과 위반 판정	W01 · W16
1	cma-builder	입력 — 자본시장 가정	W03
2	alloc-mvo	구성 — 평균분산 최적화	W04
3	alloc-bl	구성 — 블랙리터맨	W05
4	alloc-riskparity	구성 — 위험기여 균등	W06
5	ic-critic	비평·투표 — 심사와 표결	IC 전체
6	meta-reviewer	자기개선 — 예측 대 실현	W10

생략한 층은 리서치 하나뿐이다 — 새 방법을 가릴 절차를 먼저 세우는 것이 순서다

실습은 네 칸으로 적었다

코딩 경험이 없어도 따라올 수 있다

칸	무엇이 적혀 있는가	왜 필요한가
어디	어느 폴더에서 하는 일인지	엉뚱한 자리에서 치는 실수를 막는다
입력	그대로 붙여 넣을 문장이나 명령	고쳐 쓸 필요가 없다
출력	화면에 실제로 찍힌 내용	다르면 무언가 잘못된 것이다
확인	제대로 됐는지 눈으로 보는 기준	막히는 곳은 명령이 아니라 여기다

출력에 대한 단서

- 이 텍스트의 출력은 2026-08-22에 실제로 돌려 찍은 것이다
- 데이터 기간과 시장 상황이 다르면 숫자는 달라진다 — 형식이 같으면 정상이다

작업 폴더와 파이썬 환경

어디 터미널 · 아무 폴더

설치는 이 폴더 안에만 한다

입력

```
mkdir -p self-driving-mvp && cd self-driving-mvp  
python3 -m venv .venv  
.venv/bin/pip install yfinance pandas numpy scipy
```

출력

```
.venv/bin/python -c "import yfinance, pandas; print('ok')"  
ok
```

확인

폴더 안에 숨김 폴더 .venv 가 생긴다. 위 확인 명령이 ok 를 찍으면 된다.

가상환경을 쓰는 이유는 이 실습의 설치가 다른 작업을 건드리지 않게 하기 위해서다.

harness를 프로젝트에 설치한다

어디

터미널 · self-driving-mvp/

전역이 아니라 이 프로젝트에만

입력

```
git clone --depth 1 https://github.com/revfactory/harness.git /tmp/harness
mkdir -p .claude/skills
cp -r /tmp/harness/skills/harness .claude/skills/harness
```

출력

```
ls .claude/skills/harness
SKILL.md  references          ← 이 둘이 보이면 설치된 것이다

ls .claude/skills/harness/references    (6개 파일)
agent-design-patterns.md  orchestrator-template.md  ...
```

확인

Claude Code에서는 `/plugin marketplace add revfactory/harness` 로도 설치할 수 있다.

투자정책서를 쓴다

어디 편집기 · ips.md

이 파일이 파이프라인의 헌법이 된다

입력

4. 자산배분 범위
주식 30~60% · 채권 25~55% · 대체 5~25% · 개별 종목 30% 이하
5. 분산 요건
유효 종목 수($1/Sw^2$)가 4.0 이상이어야 한다.
7. 자율 파이프라인에 대한 감독
1. 산출물은 권고안이다 — 사람 IC 승인 없이 집행되지 않는다.
3. 정책 위반은 자동 기각한다 — 표결에 올리지 않는다.
4. 킬스위치 — 지시문 자동 반영을 금지한다.

출력

(저장만 한다. 아직 실행하지 않는다.)

확인

7항이 가장 중요하다 — 이 조항이 없으면 감독 지점이 사라진다.

데이터를 받는다

어디 터미널 · self-driving-mvp/

W04 실습데이터 덱의 명세를 따른다

입력

```
.venv/bin/python fetch_data.py
```

출력

저장: data/panel_monthly.csv

기간: 2015-08-01 ~ 2025-07-01 · T=120개월 · N=11자산

연율 수익률(%): SPY 14.0, EFA 7.1, EEM 6.4, IEF 1.2, TLT 0.1,
LQD 3.1, HYG 4.7, TIP 2.6, VNQ 6.8, GLD 11.6, DBC 6.2

확인

T=120, N=11 이 나오면 된다. 자산이 하나라도 빠지면 결측이 있다는 뜻이니 다시 받는다.

TLT(장기국채) 연 0.1%는 오류가 아니다 — 이 기간 금리 상승을 반영한 실제 수치다.

에이전트 팀을 만든다

어디

Claude Code · 이 폴더를 연 상태

코드를 짜지 않는다 — 팀을 설계시킨다

입력

자율주행 포트폴리오 하네스를 구성해줘. ips.md 의 제약 안에서 자본시장 가정을 만들고, MV0·블랙리터맨·리스크패리티 세 방법으로 포트폴리오를 구성한 뒤, 투자위원회 관점에서 비평·표결하고, 과거 예측과 실현 수익률을 대조해 지시문 수정을 제안하는 팀이 필요해.

출력

```
ls .claude/agents
alloc-bl.md          cma-builder.md      ips-guardian.md
alloc-mvo.md         ic-critic.md        meta-reviewer.md
alloc-riskparity.md
```

확인

파일 7개가 나오고, 하나를 열어 "## 팀 통신 프로토콜" 항목이 있으면 정상이다.

자본시장 가정을 만든다

어디

터미널 · self-driving-mvp/

평가구간은 여기서 보지 않는다

입력

```
RUN=$(date +%Y-%m-%d)
.venv/bin/python scripts/cma.py "$RUN"
```

출력

학습구간 2015-08-01 ~ 2022-07-01 (84개월), 평가구간은 보지 않았다.
Ledoit-Wolf 축소강도 0.274
기대수익률(연율 %): SPY 9.1, EFA 4.9, EEM 5.1, IEF 3.6, TLT 3.9, ...
→ runs/2026-08-22/cma.json

확인

SPY 역사적 평균 14.0%가 기대 9.1%로 줄었다 — 축소가 작동한 것이다.
84개월이어야 한다. 120개월이면 평가구간이 새어 들어간 것이다.

세 방법이 동시에 답을 낸다

어디 터미널 · self-driving-mvp/

팬아웃 — 서로의 결과를 보지 않는다

입력

```
.venv/bin/python scripts/alloc_mvo.py "$RUN"  
.venv/bin/python scripts/alloc_bl.py "$RUN"  
.venv/bin/python scripts/alloc_rp.py "$RUN"
```

출력

```
[MVO] SPY 30.0%, IEF 30.0%, TIP 25.0%, GLD 9.0%, DBC 6.0% (5종목)  
기대 5.93% · 유효N 3.93 · IPS 위반: 유효 종목 수 3.93 < 4.0 ← 주목  
[BL] SPY 9.5%, EEM 24.6%, TLT 30.0% ... (10종목)  
기대 2.33% · 유효N 5.53 · 위반 없음  
[RP] 11종목에 4.3~15.3% 분산 · 기대 5.25% · 유효N 9.92 · 위반 없음
```

확인

MVO가 다섯 종목에 몰려 분산 요건을 어겼다 — W04의 코너해가 그대로 나왔다.
사람이 찾은 것이 아니라 정책이 자동으로 잡았다는 점이 중요하다.

투자위원회가 표결한다

어디

터미널 · self-driving-mvp/

위반은 표결에 올리지 않는다

입력

```
.venv/bin/python scripts/ic_critic.py "$RUN"
```

출력

명목 목표수익률 6.5% (실질 4.5% + 인플레 2.0%)

후보 합계 관점별 점수 / 기각 사유

alloc-mvo 기각 유효 종목 수 3.93 < 4.0

alloc-bl 22.50 목표달성 3.6 위험한도 10.0 분산 6.9 집중도 2.0

alloc-riskparity 35.96 목표달성 8.1 위험한도 10.0 분산 10.0 집중도 7.9

→ 결정: 채택: Risk Parity (ERC, IPS-constrained)

확인

기각 사유가 조항과 수치로 적혀 있어야 한다 — "부적합"만으로는 감사할 수 없다.

네 관점 점수가 서로 달라야 한다. 한 관점이 압도하면 표결이 아니라 계산이다.

예측을 실현과 대조한다

어디

터미널 · self-driving-mvp/

평가구간을 여기서 처음 연다

입력

```
.venv/bin/python scripts/meta_review.py "$RUN"
```

출력

평가구간 2022-09-01 ~ 2025-07-01 (35개월)

CMA 예측오차 MAE 6.85%p (최대 오차 GLD 18.2%p)

후보	기대	실현	격차	MDD
alloc-mvo		5.93%	8.58%	+2.66%p -5.0% (기각안)
alloc-bl		2.33%	4.80%	+2.47%p -10.3%
alloc-riskparity		5.25%	7.43%	+2.18%p -7.6%

지시문 수정 제안 2건 — 자동 반영: False

확인

세 안 모두 실현이 기대를 웃돌았고, 기각된 MVO가 가장 높다.

자동 반영이 False 인지 확인한다 — True 면 킬스위치가 풀린 것이다.

한 사이클이 남긴 것

본 강의 시뮬레이션 · 2015-08~2025-07 · 자산 11개

후보	유효N	IPS	기대	실현	MDD	표결
MVO (W04)	3.93	위반	5.93%	8.58%	-5.0%	기각
블랙리터맨 (W05)	5.53	적격	2.33%	4.80%	-10.3%	22.50
리스크 패리티 (W06)	9.92	적격	5.25%	7.43%	-7.6%	35.96 채택

읽을 때 주의할 것

- 한 구간 한 번의 결과다. 방법의 우열을 말해 주지 않는다
- 실현이 기대를 웃돈 것은 이 기간 시장이 좋았기 때문이다 — 예측이 맞아서가 아니다

규율이 가장 좋은 안을 걸렀다

이 세션에서 가장 불편한 결과

일어난 일

- MVO는 분산 요건 하나 때문에 기각됐다 ($3.93 < 4.0$)
- 사후적으로 MVO의 실현수익이 8.58%로 가장 높았다
- MDD도 -5.0%로 가장 얇았다

그래서 규칙을 바꿔야 하는가

- 기각 근거는 사전에 정한 정책이었다
- 결과를 보고 기준을 고치면 정책이 아니라 사후 합리화가 된다
- 그러나 4.0이라는 값 자체는 검토 대상이다

이 물음이 이 세션의 IC 케이스 안건이다

규율을 지킨 대가를 수치로 본 상태에서 논의를 시작한다는 점이 이 케이스의 조건이다.

가정은 크게 틀렸다

그런데도 결과는 나쁘지 않았다

자산	CMA 기대	실현	오차
GLD (금)	6.4%	24.6%	+18.2%p
전체 평균 절대오차	—	—	6.85%p

무엇을 뜻하는가

- 기대수익률 추정은 이 정도로 틀린다 — 이것이 정상 범위다
- 그래서 리스크 패리티처럼 기대수익률에 덜 의존하는 방법이 있는 것이다 (W06)
- 성과가 괜찮았던 것은 예측이 맞아서가 아니라 제약이 분산을 강제했기 때문이다

가정의 정확도보다 가정이 틀렸을 때 건디는 구조가 중요하다.

기계는 스스로 고치겠다고 했다

그리고 우리는 막았다

meta-reviewer의 제안 (runs/2026-08-22/meta_review.json)

지시문 수정 제안 2건 — 자동 반영: False

- [cma-builder] SHRINK_MU 0.5 → 0.7 (역사적 평균 의존을 낮춘다)
- [cma-builder] GLD의 기대수익률 산정에 국면 구분을 추가한다

"gate": "IPS 7.4항 — 사람이 변경 이력을 검토한 뒤에만 반영한다"

두 제안은 타당한가

- 첫째는 합리적으로 보인다 — 예측오차가 컸으니 역사적 평균 의존을 줄이자는 것이다
- 그러나 한 구간의 오차로 축소 비율을 올리면, 다음 구간에 반대로 틀릴 수 있다
- 판단은 사람이 한다. 이 지점이 킬스위치가 지키는 자리다

이 구조가 안고 있는 위험

CFA Institute의 지적과 우리의 대응

지적	내용	MVP에서의 대응
연쇄 오류	한 에이전트의 오류가 하류로 전파된다	단계마다 JSON으로 남겨 추적
거짓 합의	리뷰어가 반박보다 동조로 기운다	규칙 기반 자동 기각을 앞에 둠
자기개선의 역설	지시문이 조금 바뀌어도 행동이 크게 흔들린다	킬스위치 — 자동 반영 금지
감사 가능성	50개 에이전트와 계속되는 코드 수정	에이전트 7개로 축소 · 전 과정 기록
국면 민감성	안정기엔 잘 하고 고변동성에서 저하된다	대응하지 못했다

마지막 항목은 우리도 풀지 못했다 — GLD 18.2%p 오차가 그 증거다

사람은 무엇을 계속 해야 하는가

수동적 안전장치로는 아직 충분하지 않다

이번 사이클에서 사람이 실제로 개입한 지점

- 정책을 썼다 — 분산 요건 4.0을 정한 것은 사람이다
- 기각을 받아들였다 — 성과가 좋아 보이는 안을 규율에 따라 버렸다
- 제안을 보류했다 — 메타의 지시문 수정을 반영하지 않았다
- 가정을 의심했다 — 예측오차 6.85%p를 정상 범위로 읽을지 판단했다

"AI 시스템은 시장이 가장 불안할 때 가장 덜 신뢰할 만하다" — CFA Institute

자동화가 진행될수록 사람의 일은 줄지 않고 옮겨 간다. 계산에서 정책과 판단으로 옮겨 간다.

다음 사이클을 돌려 본다

제출물은 의결문 한 장이다

#	무엇을 한다	제출
1	ips.md의 분산 요건을 4.0에서 3.5로 낮추고 다시 돌린다	표결 결과 변화
2	CMA의 SHRINK_MU를 0.7로 올려 돌리고 예측오차를 비교한다	MAE 변화
3	ic-critic에 '견고성' 관점을 추가해 표결을 다시 한다	점수표
4	위 세 변경 가운데 무엇을 채택할지 의결문으로 쓴다	A4 1장

의결문에 반드시 담을 것

- 무엇을 근거로 정책을 바꾸었는가 — 또는 바꾸지 않았는가
- 그 판단이 사후 결과에 이끌린 것은 아닌지에 대한 자기 점검

참고문헌

구분	출처
논문	Ang, A., Azimbayev, N., & Kim, A. (2026). The Self Driving Portfolio: Agentic Architecture for Institutional Asset Management. arXiv:2604.02279
비평	CFA Institute Enterprising Investor (2026). Self-Driving Portfolio: Promise and Pitfalls
도구	revfactory/harness — 황민호. Apache-2.0. github.com/revfactory/harness
데이터	Yahoo Finance (yfinance) — 월간 총수익률, 배당 포함
교재	본 과정 W03 · W04 · W05 · W06 · W10 · W16 및 각 주차 실습데이터 덱

이 세션의 실습 코드와 산출물은 XS_자율주행포트폴리오/self-driving-mvp/ 에 있다. 덱의 모든 출력은 2026-08-22 에 실제로 실행해 얻은 것이다.